

Appendix: Full Per-Run Tables

This appendix holds the exhaustive per-run tables the body reports summaries from, so the paper can report analysis without forcing every reader through raw run-by-run data. Every number here is read directly from a file under `experiments/results/`; the source path is given with each table so a reviewer can check it without rerunning anything. Reproduction instructions are in `docs/reproducibility.md` and the full claim-to-evidence mapping is in `docs/claim_evidence_map.md`. In the tables below, KS is Kafka Streams 4.3.1 and Flink is Flink 2.2.0; workload codes W1 through W5 are defined in A2.

A1. Correctness, baseline and repeat

Source: `experiments/results/engine_correctness_summary.md`. Each cell is `expected/actual`; all 20 rows report zero missing, zero unexpected, and zero duplicate records.

Engine	Workload	Baseline	repeat1
Flink	W1	1000/1000	1000/1000
Flink	W2	513/513	513/513
Flink	W3	199/199	199/199
Flink	W4	1099/1099	1099/1099
Flink	W5	11024/11024	11024/11024
KS	W1	1000/1000	1000/1000
KS	W2	513/513	513/513
KS	W3	199/199	199/199
KS	W4	1099/1099	1099/1099
KS	W5	11024/11024	11024/11024

A2. Per-workload run metadata

All runs use 100 keys and seed 7, `apache/kafka:4.3.1` in KRaft mode, Kafka Streams 4.3.1 with `exactly_once_v2`, and Flink 2.2.0 with `flink-connector-kafka` 5.0.0-2.2 and `EXACTLY_ONCE` sink delivery. Both engines consume with `read_committed` isolation.

Workload	Events	Producer records	Left/right records	Start ms	Notes
W1 identity	1000	1000	-	0	One output per input.

Workload	Events	Producer records	Left/right records	Start ms	Notes
W2 fil- ter_map	1000	1000	-	0	513 expected outputs (odd payloads filtered, even payloads dou- bled).
W3 tum- bling_count	1000	1001	-	0	+1 producer- only __tick__ record closes the final window; 199 expected outputs.
W4 slid- ing_sum	1000	1001	-	600000	Start offset avoids negative window- start times- tamps in native hopping windows; 1099 expected outputs.

Workload	Events	Producer records	Left/right records	Start ms	Notes
W5 stream_stream_join	-	-	1000 / 1000	1000	Start offset avoids negative right-side times-tamps; 11024 expected outputs.

A3. Fixed-rate proxy latency, 10/20/40 records per second

Source: `experiments/results/latency_summary.md`. Measurement: host write to read-committed-visible-read proxy (`t0` to `t3`), 100-input runs (W3-W5 add one producer-only tick and time each output from its latest contributing input event). All 40 rows report zero missing, zero unexpected, zero duplicate records against their workload's expected count.

Engine	Workload	Rate	Expected	p50 ms	p95 ms	p99 ms
KS	W1	10/sec	100	1022.525	2334.453	2734.456
KS	W1	20/sec	100	1332.012	2631.678	2831.634
		(baseline)				
KS	W1	20/sec	100	1226.389	2633.999	2833.992
		(repeat1)				
KS	W1	40/sec	100	1704.546	2828.505	2928.505
Flink	W1	10/sec	100	1260.707	2060.664	2460.697
Flink	W1	20/sec	100	951.127	2393.965	2593.946
		(baseline)				
Flink	W1	20/sec	100	948.056	2447.962	2647.962
		(repeat1)				
Flink	W1	40/sec	100	1990.019	3114.001	3214.316
KS	W2	10/sec	56	1221.640	2526.477	2825.823
KS	W2	20/sec	56	1226.708	2843.058	2993.158
		(baseline)				
KS	W2	20/sec	56	1265.364	2665.192	2814.333
		(repeat1)				
KS	W2	40/sec	56	1765.886	2890.619	2964.982
Flink	W2	10/sec	56	1332.854	2332.809	2632.060
Flink	W2	20/sec	56	1400.690	3200.614	3350.702
		(baseline)				

Engine	Workload	Rate	Expected	p50 ms	p95 ms	p99 ms
Flink	W2	20/sec	56	900.849	2611.810	2761.075
		(repeat1)				
Flink	W2	40/sec	56	2088.450	3213.160	3287.256
KS	W3	10/sec	71	5331.984	11132.103	11631.267
KS	W3	20/sec	71	3973.097	6873.202	7123.280
		(baseline)				
KS	W3	20/sec	71	3595.147	6495.271	6744.307
		(repeat1)				
KS	W3	40/sec	71	2368.306	3818.504	3942.612
Flink	W3	10/sec	71	4228.258	10028.509	10527.388
Flink	W3	20/sec	71	2167.446	5068.230	5317.375
		(baseline)				
Flink	W3	20/sec	71	2509.091	5409.743	5658.188
		(repeat1)				
Flink	W3	40/sec	71	2077.073	3527.950	3651.222
KS	W4	10/sec	710	4865.798	10665.881	11164.941
KS	W4	20/sec	710	3964.038	6864.071	7113.249
		(baseline)				
KS	W4	20/sec	710	3191.806	6091.807	6338.270
		(repeat1)				
KS	W4	40/sec	710	2433.497	3883.589	4006.722
Flink	W4	10/sec	710	4091.697	9891.631	10390.270
Flink	W4	20/sec	710	2638.442	5538.424	5786.186
		(baseline)				
Flink	W4	20/sec	710	2359.980	5259.953	5507.883
		(repeat1)				
Flink	W4	40/sec	710	1965.928	3415.926	3540.110
KS	W5	10/sec	186	1835.608	3035.445	3435.417
KS	W5	20/sec	186	2544.645	3896.173	4096.137
		(baseline)				
KS	W5	20/sec	186	2540.653	3898.778	4098.726
		(repeat1)				
KS	W5	40/sec	186	2195.370	3338.270	3438.284
Flink	W5	10/sec	186	1135.848	2039.220	2439.144
Flink	W5	20/sec	186	805.600	2321.768	2521.762
		(baseline)				
Flink	W5	20/sec	186	1547.947	3059.302	3259.235
		(repeat1)				
Flink	W5	40/sec	186	1531.194	2705.081	2805.058

A4. Long-duration stability, 100 events/sec

Source: experiments/results/*_latency_stability_100/latency_summary.json and run_metadata.json. Kafka Streams W1-W4 and Flink W1-W4 all ran

180,000 events (30 minutes). W5 stability uses a bounded 120-second, 12,000-event run for both engines instead of the 30-minute/180,000-event parameters used for W1-W4: `stream_stream_join`'s expected-output cardinality grows worse than linearly with input volume under a 10-minute join window (a 1000x1000-event run already produces 11,024 outputs; the bounded 12,000-event run produced 1,093,417), and an earlier 180,000-event attempt at these window parameters produced a 3.2 GB expected-output file and did not finish after 11 hours. Getting clean Flink W3/W4 numbers required fixing two bugs in `scripts/run-flink-w1-latency.sh` (a stale-checkpoint crash and a too-short consumer idle timeout); the first attempts both returned zero matched records for reasons unrelated to engine behavior. See Section 7 for both discussions. The `t1-t0`, `t2-t1`, and `t3-t2` columns are the calibrated broker-ingestion, engine-processing, and downstream-visibility hops (Section 4.2). Backlog/consumer-lag CSVs for each run are at `experiments/results/*_latency_stability_100/lag.csv`.

Engine	WL	Dur	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
KS	W1	30 min	180000	1216.3	1967.4	1000.7	6.0	1008.6
KS	W2	30 min	89906	999.4	1928.3	1000.5	5.0	1009.3
KS	W3	30 min	29924	2876.1	6576.4	991.9	6083.0	19.0
KS	W4	30 min	30900	2926.2	6813.6	999.2	6249.0	65.4
KS	W5	2 min (bounded)	1093417	2172.1	2913.5	1001.4	965.0	1107.5
Flink	W1	30 min	180000	1012.7	1882.1	1000.3	4.0	1001.0
Flink	W2	30 min	89906	1007.8	1874.9	999.9	4.0	997.0
Flink	W3	30 min	29924	1789.3	5506.7	991.7	4287.0	968.6
Flink	W4	30 min	30900	1840.8	5751.4	1000.4	5080.0	994.9

A5. Fault injection, W1 identity, 20 records/sec, 2000 events, failure injected at 50 seconds

Source: `experiments/results/*_identity_latency_failure*/latency_summary.json` and `experiments/results/failure_latency_aggregated.csv`. Failure mechanics (`scripts/run-failure-test.sh`): `jvm_kill` sends SIGKILL to the worker container and restarts the same container 5 seconds later (local state

volume intact); **broker_kill** stops and restarts the Kafka container 5 seconds later; **node_loss** removes the worker container and its anonymous volumes (`docker compose rm -fsv`) and recreates it fresh 5 seconds later, forcing full local-state loss. All six runs matched 2000/2000 expected output records with zero missing, zero unexpected, zero duplicates, so no failure caused an externally visible correctness violation, only a latency spike. Kafka Streams **node_loss** needed a corrected retry after a script bug made the first attempt rejoin the wrong consumer group; the number below is from the corrected run.

Engine	Failure mode	p99 total ms	p99 t1-t0 ms	p99 t2-t1 ms	p99 t3-t2 ms
KS	jvm_kill	43976.7	1381.3	42944.0	1019.2
KS	broker_kill	9892.3	8455.0	108.0	9387.3
KS	node_loss	44225.0	1574.6	43240.0	960.6
Flink	jvm_kill	7557.5	1922.5	7232.0	996.4
Flink	broker_kill	9743.4	8783.1	408.0	1009.8
Flink	node_loss	9208.6	1613.7	8931.0	1062.1

A6. Tuning matrix

Source: `experiments/results/*_w3_latency_tuning*/latency_summary.json` and `experiments/results/*_w3_latency_tuning_control/latency_summary.json`. All rows use W3 tumbling_count, 100 input records plus one tick, 20 records/sec, so only the commit/checkpoint interval varies between the control and tuning rows for a given engine. All four rows matched 71/71 expected records with zero missing, unexpected, or duplicate records. Raising the interval 10x moves a different hop per engine: Kafka Streams' engine-processing hop (**t2-t1**) rises 6.1x while its commit hop stays flat, consistent with the commit interval gating when its `suppress(untilWindowCloses())` buffer is released; Flink's processing hop barely moves while its commit hop rises 10.9x, consistent with its **EXACTLY_ONCE** Kafka sink committing on a one-checkpoint lag. Section 6 reads both rows together.

Engine	Interval	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
KS	1000 ms (control)	71	3080.8	6230.0	2469.9	3715.0	60.1
KS	10000 ms	71	21946.2	25095.5	2439.7	22616.0	57.5

Engine	Interval	Matched	p50 ms	p99 ms	p99 t1-t0	p99 t2-t1	p99 t3-t2
Flink	1000 ms (control)	71	2293.7	5442.5	2539.6	2655.0	271.8
Flink	10000 ms	71	4993.1	8142.2	2486.7	2728.0	2954.5

A7. Resource utilization

Source: `experiments/results/resource_metrics/*.csv`, sampled via `docker stats` at fixed intervals against the running benchmark containers (`src/stream_state_bench/resource_monitor.py`). This is a shared-host, cgroup-level sample, not an isolated hardware profile (Section 7). The two engines' rows come from different collection windows and are not directly comparable as a controlled experiment: Kafka Streams rows are short (roughly 1-2 minute), matched-duration bursts, one per workload, sampled every 5-10 seconds; the Flink row is a single blended average across its entire ~2-hour, four-workload 30-minute stability sweep, sampled every 15 seconds, and cannot be separated back out per workload from this file alone.

Engine	Container	WL / win- dow	Samples	CPU mean	CPU max	Mem mean	Mem max
KS	worker	W1 (burst)	8	2.76%	6.50%	117.5 MiB	131.3 MiB
KS	worker	W2 (burst)	13	1.89%	6.03%	122.1 MiB	130.5 MiB
KS	worker	W3 (burst)	12	2.70%	16.07%	142.9 MiB	147.6 MiB
KS	worker	W4 (burst)	13	4.64%	43.25%	157.0 MiB	161.3 MiB
KS	worker	W5 (2 min bounded)	16	10.68%	41.45%	219.7 MiB	404.8 MiB
KS	broker	W5 (2 min bounded)	16	60.24%	288.92%	455.7 MiB	887.6 MiB
Flink	worker	W1- W4 blended (~2 h)	841	2.59%	300.50%	377.6 MiB	565.0 MiB

Engine	Container	WL / win- low	Samples	CPU mean	CPU max	Mem mean	Mem max
Flink	broker	W1- W4 blended (~2 h)	841	49.86%	331.31%	675.9 MiB	1073.2 MiB